PROJE: Sayı Sistemleri ve Algoritmalar

DERS: BMG

AD: Nail Buğra
SOYAD: Önür

ÖĞRENCİ NO: 24360859010

GİRİŞ

Bu projede bilgisayar bilimlerinin temel kavramlarından biri olan sayı sistemlerini ve algoritmaları ele aldım. Amacım, bit ve byte kavramlarını, binary–decimal–hexadecimal sistemlerini ve Two's Complement yöntemini açıklamak, ardından Python ile taban dönüşümünü uygulamalı olarak göstermek. Böylece hem teorik hem pratik bir bakış açısı sunmayı hedefledim.

Projenin Amacı:

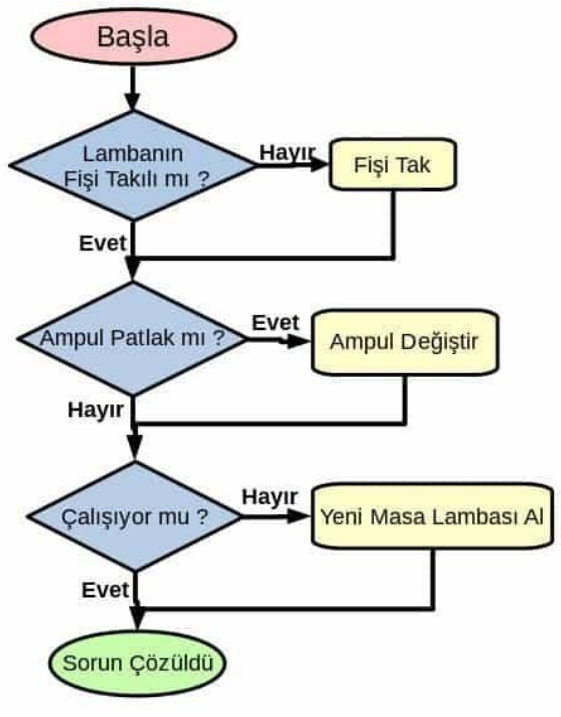
*Sayı sistemlerini tanıtmak

*Algoritmaların önemini göstermek

* Python uygulaması ile pratik yapmak

Algoritmaların Önemi:

*Günlük hayat



*Bilgisayar bilimleri

*Problem çözme

BİT VE BYTE

Bit Kavramı

En Küçük Birim: Bit, dijital dünyadaki en küçük bilgi birimidir.

Fiziksel Karşılığı: Bilgisayar donanımında bu durumlar genellikle voltajla temsil edilir:

* 0: Kapalı, Akım Yok, Yanlış (False)

* 1: Açık, Akım Var, Doğru (True)

Kombinasyon Gücü: Bit sayısı arttıkça temsil edilebilecek durum sayısı logaritmik olarak artar (2^n formülü ile).

* *Örnek:* 2 bit ile 4 farklı durum (00, 01, 10, 11) ifade edilebilir.

```
┌──────────────────────────────────┐
│        BİT KOMBİNASYON GÜCÜ      │
├──────────────────────────────────┤
│  1 Bit = 2¹ = 2 durum            │
│  ┌───┬───┐                       │
│  │ 0 │ 1 │                       │
│  └───┴───┘                       │
│                                  │
│  2 Bit = 2² = 4 durum            │
│  ┌───┬───┬───┬───┐               │
│  │00 │01 │10 │11 │               │
│  └───┴───┴───┴───┘               │
│                                  │
│  3 Bit = 2³ = 8 durum            │
│  000, 001, 010, 011,             │
│  100, 101, 110, 111              │
│                                  │
│  8 Bit = 2⁸ = 256 durum          │
│  (1 Byte - tüm ASCII karakterler) │
│                                  │
│  FORMÜL: n bit → 2ⁿ kombinasyon  │
└──────────────────────────────────┘
```

Byte Kavramı

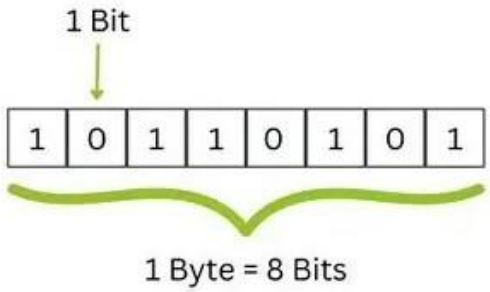
Tanım: Standart olarak 8 bitin yan yana gelmesiyle oluşan birimdir.

Kapasite: 8 bitten oluştuğu için toplamda **256 farklı değer** (2^8 = 256) alabilir. Bu kapasite, alfabedeki tüm harfleri, sayıları ve temel işaretleri tanımlamak için yeterlidir.

ASCII ve Karakterler: Örneğin, klavyeden "A" harfine bastığınızda, bilgisayar bunu arka planda `01000001` şeklinde bir byte (8 bit) olarak işler.

Ölçü Birimi Başlangıcı: Dosya boyutları bitten ziyade genellikle Byte ve katları ile ifade edilir.

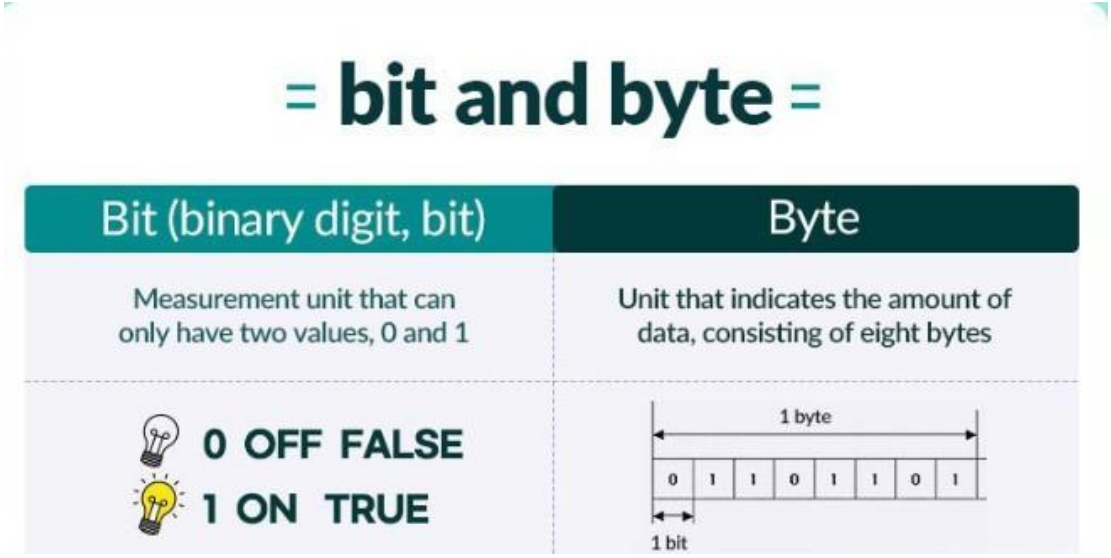
* 1024 Byte = 1 Kilobyte (KB)
* 1024 KB = 1 Megabyte (MB)




```
┌──────────────────────────────────────────┐
│              BYTE (B) - TEMEL TANIM      │
├──────────────────────────────────────────┤
│                                          │
│  ▌  8 BİT'ten oluşan standart veri birimi ▌ │
│                                          │
│                                          │
│  FORMÜL: 1 Byte = 8 Bit                  │
│          2⁸ = 256 farklı değer           │
│          Değer aralığı: 0 - 255          │
│                                          │
│  ╔═══════════════════════════════════╗   │
│  ║  1 Byte = [1][0][1][1][0][0][0][1] = 177  ║ │
│  ║   128+0+32+16+0+0+0+1 = 177          ║ │
│  ╚═══════════════════════════════════╝   │
│                                          │
│  KULLANIM:                               │
│  • 1 karakter (ASCII/UTF-8)              │
│  • Bellek adresleme temel birimi         │
│  • Dosya boyutu ölçümü                   │
└──────────────────────────────────────────┘
```

Bit ve Byte Karşılaştırması

Özellik	Bit (b)	Byte (B)
İçerik	En küçük ikili birim (0 veya 1)	8 adet bitin birleşimi
Sembol	Küçük "b" harfi ile gösterilir	Büyük "B" harfi ile gösterilir
Kullanım Alanı	Genelde **İnternet hızı** (Mbps)	Genelde **Dosya boyutu** (MB, GB)



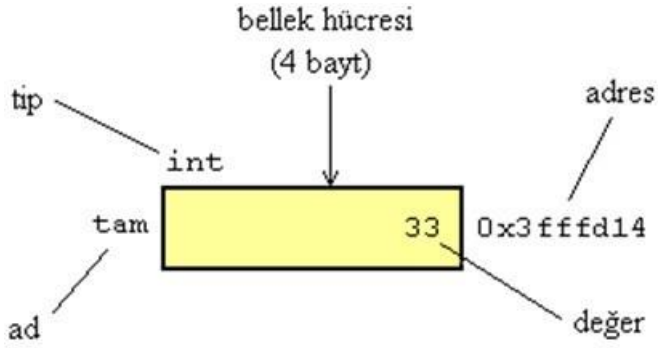

```
┌──────────────────┬──────────────────┬──────────────────┐
│     KATEGORİ     │       BİT        │       BYTE       │
├──────────────────┼──────────────────┼──────────────────┤
│  İnternet Hızı   │ Mbps, Gbps       │ MB/s, GB/s       │
│                  │ (Mega/GigaBIT)   │ (Mega/GigaBYTE)  │
│                  │ Örn: 100 Mbps    │ Örn: 12.5 MB/s   │
├──────────────────┼──────────────────┼──────────────────┤
│  Depolama        │ Bit cinsinden    │ Byte cinsinden   │
│                  │ kapasite         │ kapasite         │
│                  │ Örn: 1 TB =      │ Örn: 1 TB =      │
│                  │ 8.796e12 bit     │ 1.0995e12 byte   │
├──────────────────┼──────────────────┼──────────────────┤
│  Bellek          │ Bit adresleme    │ Byte adresleme   │
│                  │ Örn: RAM bitleri │ Örn: 4GB RAM     │
├──────────────────┼──────────────────┼──────────────────┤
│  Veri Transferi  │ Seri iletişim    │ Paralel iletişim │
│                  │ (bit-bit)        │ (byte-byte)      │
└──────────────────┴──────────────────┴──────────────────┘

‼ DÖNÜŞÜM: 1 Byte = 8 Bit
  100 Mbps internet ≈ 12.5 MB/s indirme hızı
```

Kullanım Alanları

Depolama:

Bellek adresleme:



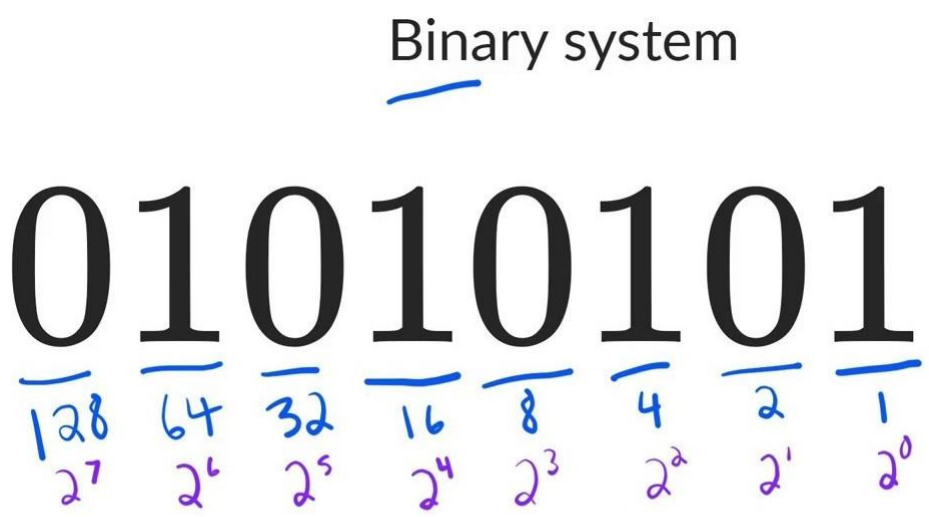
Veri aktarımı:

SAYI SİSTEMLERİ

Binary (2'lik sistem): Bilgisayarların doğal dili.

*** 2 tabanlı sistem (0,1)**

*** Örnek: 1010(2) = 10(10)**



Decimal (10'luk sistem): Günlük hayatta kullandığımız sistem.

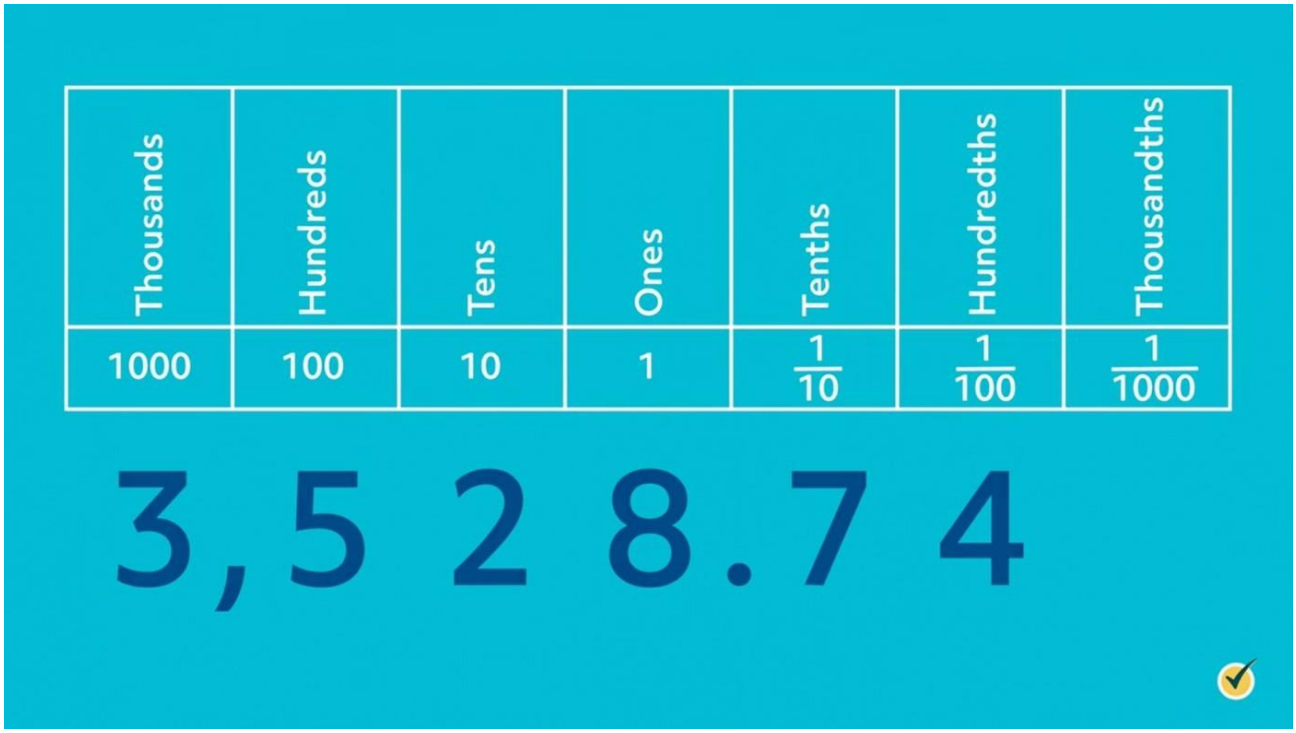
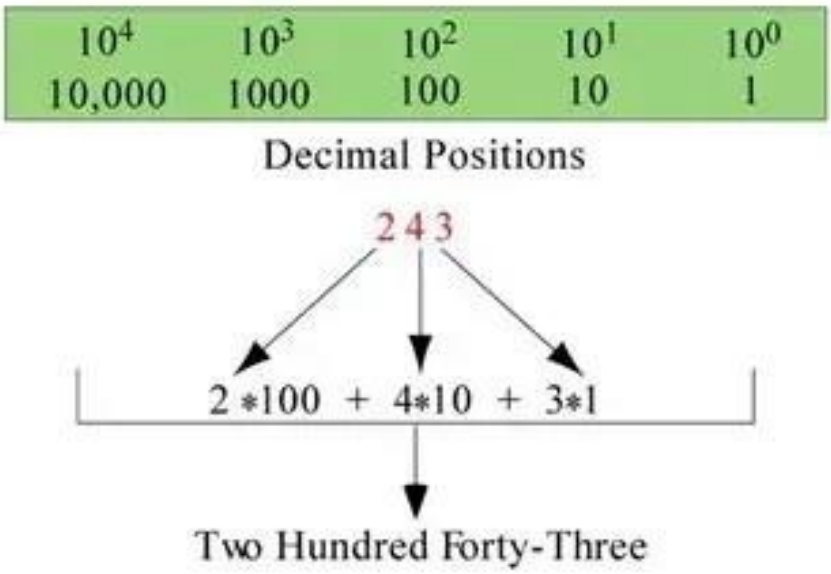


Figure 3-1

Decimal system



Hexadecimal (Onaltılık sistem): Programlama, bellek adreslemede ve renk kodlarında sıkça kullanılır.
Örnek: Binary `1010` → Decimal `10` → Hexadecimal A.

Color Name	RGB CODE	HEX #	Sample
Pale Goldenrod	238-232-170	eee8aa	
Light Goldenrod Yellow	250-250-210	fafad2	
Light Yellow	255-255-224	ffffe0	
Yellow	255-255-0	ffff00	
Gold	255-215-0	ffd700	
Light Goldenrod	238-221-130	eedd82	
Goldenrod	218-165-32	daa520	
Dark Goldenrod	184-134-11	b8860b	

Sayı Sistemleri Arası Dönüşüm:

Dönüşüm Örnekleri

*** 1) Binary -> Decinal(örnek: 1101 -> 13)**

*** 2) Decimal ->Hexadecimal (örnek: 255 -> FF)**

1) örnek: 1101 -> 13

```
┌──────────────────────────────────────┐
│   BINARY → DECIMAL DÖNÜŞÜMÜ          │
│         Örnek: 1101₂ → 13₁₀          │
├──────────────────────────────────────┤
│                                      │
│  Binary: 1   1   0   1               │
│  Pozisyon: 3   2   1   0             │
│  Değer:  2³  2²  2¹  2⁰              │
│          8   4   2   1               │
│                                      │
│  HESAPLAMA:                          │
│  1 × 2³ = 1 × 8 = 8                  │
│  1 × 2² = 1 × 4 = 4                  │
│  0 × 2¹ = 0 × 2 = 0                  │
│  1 × 2⁰ = 1 × 1 = 1                  │
│                                      │
│  TOPLAM: 8 + 4 + 0 + 1 = 13 ✓        │
│                                      │
│  FORMÜL: ∑(bit × 2ⁿ)                 │
│  n = bit pozisyonu (sağdan 0)        │
└──────────────────────────────────────┘
```

2) örnek: 255 -> FF

```
|   DECIMAL → HEXADECIMAL DÖNÜŞÜMÜ          |
|          Örnek: 255₁₀ → FF₁₆              |
|-------------------------------------------|
|                                           |
|  Adım 1: 255 ÷ 16 = 15 kalan 15           |
|  Adım 2: 15 ÷ 16 = 0 kalan 15             |
|                                           |
|  Kalanları ters sırada yaz: 15 15         |
|                                           |
|  HEX TABLOSU:                             |
|  10 → A      13 → D                       |
|  11 → B      14 → E                       |
|  12 → C      15 → F                       |
|                                           |
|  15 = F, 15 = F → FF₁₆ ✓                  |
|                                           |
|  KONTROL:                                 |
|  F₁₆ = 15₁₀                               |
|  FF₁₆ = (15×16) + (15×1) = 240+15=255     |
|                                           |
|  FORMÜL: Sayıyı 16'ya böl, kalanları      |
|          ters sırada hex'e çevir.         |
```

Karşılaştırma Tablosu

Sistem	Taban	Rakamlar	Kullanım Alanı
Binary	2	0, 1	Bilgisayar işlemleri
Decimal	10	0–9	Günlük hayat
Hex	16	0–9, A–F	Bellek adresleme, renk

Hexadecimal Number System Table

Decimal	Binary	Hexa-Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

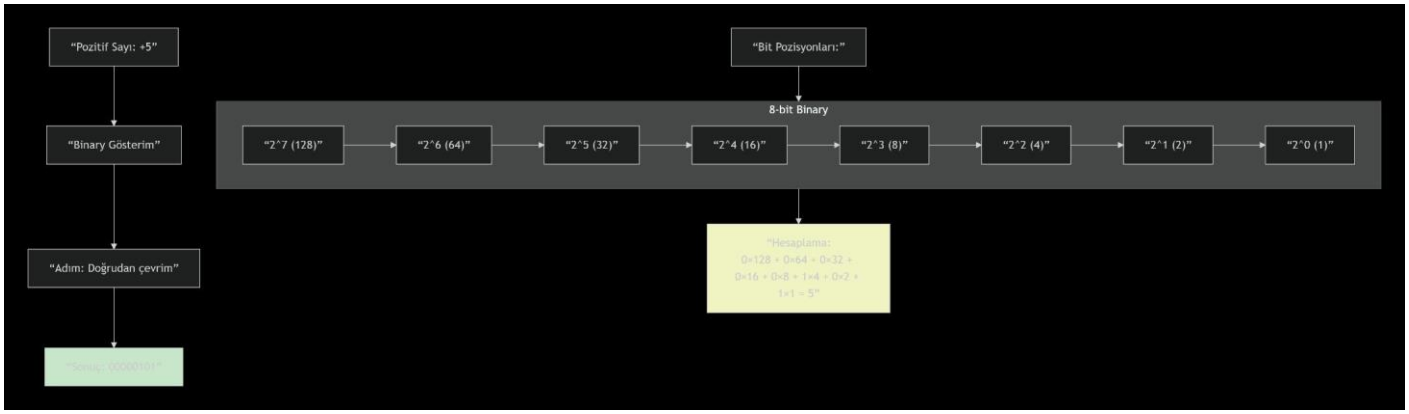
TWO’S COMPLEMENT

Pozitif Sayılar:

Pozitif Sayıların Binary Gösterimi

*** Normal binary gösterim**

*** Örnek: +5 -> 00000101 (8 bit)**



Bit	Değer	Güç (2^n)	+5 Örneği
7	128	2^7	0
6	64	2^6	0
5	32	2^5	0
4	16	2^4	0
3	8	2^3	1
2	4	2^2	0
1	2	2^1	1
0	1	2^0	1

HESAPLAMA (+5 için):

0×128 + 0×64 + 0×32 + 0×16 + 1×8 + 0×4 + 1×2 + 1×1 = 5

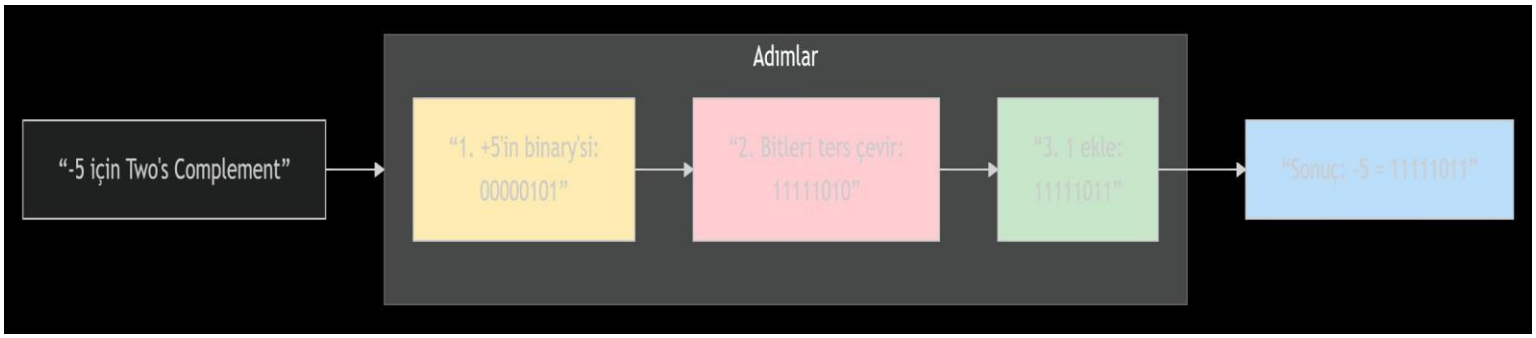
Negatif Sayılar:

Two’s Complement ile Negatif Sayıları

*** 1) Tüm bitleri ters çevir.**

*** 2) 1 ekle**

*** Örnek: -5 -> 11111011**



Bit	Değer	Açıklama	-5 Örneği
7	-128	İşaret biti	1
6	+64		1
5	+32		1
4	+16		1
3	+8		1
2	+4		0
1	+2		1
0	+1		1

HESAPLAMA (-5 için):

(-128) + 64 + 32 + 16 + 8 + 0 + 2 + 1 = -5

Avantajları:

Neden Two’s Complement?

* Matematiksel işlemlerde kolaylık (toplama ile çıkarma yapılabilir)

```
NEDEN TWO'S COMPLEMENT?
Çünkü: TOPLAMA = ÇIKARMA
──────────────────────────────────────────────

ÖRNEK: 5 - 3 = 2 işlemi

Normal:   5 - 3 = 2
Two's Complement ile:

5  = 00000101
-3 = 11111101  (Two's Complement of 3)
────────────────────
Toplam: 100000010

Taşan ilk bit atılır:
00000010 = 2 ✓

AVANTAJLARI:
1. Çıkarma = Toplama + Two's Complement
2. Tek devre yeterli (+ ve - için)
3. Sıfır için tek gösterim (00000000)
4. -128..+127 aralığı simetrik
```

* Donanımda basit uygulama (tek devre)

```
┌──────────────────────────────────────┐
│ NEDEN TWO'S COMPLEMENT?              │
│ Donanımda Basit Uygulama (Tek Devre) │
├──────────────────────────────────────┤
│                                      │
│ NORMAL SİSTEM:                       │
│ Toplama → Toplayıcı Devre            │
│ Çıkarma → Çıkarıcı Devre             │
│ (2 AYRI DEVRRE GEREKİR)              │
│                                      │
│ TWO'S COMPLEMENT:                    │
│ Toplama → Toplayıcı Devre            │
│ Çıkarma → Toplayıcı Devre + Complement │
│ (TEK DEVRRE YETERLİ!)                │
│                                      │
│ DEVRRE YAPISI:                       │
│ ┌──────────────┐                     │
│ │  TOPLAYICI   │                     │
│ │   DEVRESİ    │                     │
│ └──────┬───────┘                     │
│        │                             │
│ ┌──────▼──────┐                      │
│ │Two's Complement │                  │
│ │   (XOR + 1)  │                     │
│ └──────────────┘                     │
│                                      │
│ AVANTAJ:                             │
│ • Daha az transistör                 │
│ • Daha az güç tüketimi               │
│ • Daha hızlı işlem                   │
│ • Daha ucuz üretim                   │
│                                      │
```

Ek Bilgiler:

* MSB (Most Significant Bit): En soldaki bit
    * 0 ise -> pozitif sayı
    * 1 ise -> negatif sayı

* 8-bit aralığı: -128 ile +127 arası

* Özel durum: -128’in pozitif karşılığı yok (10000000)

PYTHON KOD UYGULAMASI

Kod Mantığı:

Python ile Dönüşüm Uygulaması

*** Girdi -> Dönüşüm -> Çıktı**

*** Kullanıcıdan decimal sayı alır.**

*** Binary , Hex ve Two’s Complement çıktısı verir.**

```
┌──────────────────────────────────────────────────────────┐
│          AKIŞ ŞEMASI - SAYI DÖNÜŞÜM PROGRAMI             │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  [BAŞLA] → [SAYİ GİR] → [POZİTİF/NEGATİF KONTROL]        │
│       ↓                     ↓            ↓               │
│       ↓               [POZİTİF İSE] [NEGATİF İSE]        │
│       ↓                     ↓            ↓               │
│       ↓             [BİNARY ÇEVİR] [2's COMP HESAPLA]    │
│       ↓                     ↓            ↓               │
│       └──────────→ [HEX ÇEVİR] ←────────┘                │
│                             ↓                            │
│                     [SONUÇLARI YAZDIR]                   │
│                             ↓                            │
│                     [TEKRAR SOR] → [BİTTİ]               │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Kod Parçaları:

Önemli Fonksiyonlar

```python
def to_hex(n: int) -> str:
    """Pozitif onluk sayıyı hex stringe çevirir."""
    digits = "0123456789ABCDEF"
    if n == 0:
        return "0"
    result = ""
    while n > 0:
        result = digits[n % 16] + result
        n //= 16
    return result
```

```python
def twos_complement_8bit(n: int) -> str:
    """Girilen sayıyı 8-bit two's complement olarak döndürür."""
    if n < -128 or n > 127:
        return "Hata: 8-bit aralığı dışı (-128..127)"

    if n >= 0:
        return bin(n)[2:].zfill(8)

    # Negatif sayı için two's complement
    return bin((1 << 8) + n)[2:].zfill(8)
```

```python
def to_binary(n: int) -> str:
    """Pozitif onluk sayıyı binary stringe çevirir."""
    if n == 0:
        return "0"
    bits = ""
    while n > 0:
        bits = str(n % 2) + bits
        n //= 2
    return bits
```

Akış Diyagramı:

Programın işleyişi

1. Kullanıcıdan sayı al.
2. Sayıyı binary, hex ve two's complement'e çevir.
3. Sonuçları ekrana yazdır.

1. Kullanıcıdan sayı al

```
try:
    n = int(input("Onluk tabanda bir sayi giriniz (-128 ile 127 arasi): "))
    choice = input("Binary için B, Hex için H yaziniz: ").upper()
```

2. Sayıyı binary'e çevir

```
def to_binary(n: int) -> str:
    """Pozitif onluk sayiyi binary stringe çevirir."""
    if n == 0:
        return "0"
    bits = ""
    while n > 0:
        bits = str(n % 2) + bits
        n //= 2
    return bits
```

2. Sayıyı hex’e çevir

```python
def to_hex(n: int) -> str:
    """Pozitif onluk sayiyi hex stringe çevirir."""
    if n == 0:
        return "0"
    digits = "0123456789ABCDEF"
    result = ""
    while n > 0:
        result = digits[n % 16] + result
        n //= 16
    return result
```

2. Sayıyı two’s complement’e çevir

```python
def pad_bits(bitstring: str, width: int = 8) -> str:
    """Bit dizisini width uzunluğa sifirlarla doldurur."""
    return bitstring.zfill(width)

def twos_complement_8bit(n: int) -> str:
    """Girilen sayiyi 8-bit two's complement olarak döndürür."""
    if n < -128 or n > 127:
        return "Hata: 8-bit araliği dişi (-128..127)"
    if n >= 0:
        return pad_bits(to_binary(n), 8)

    # Negatif için: |n| binary → tersle → +1
    abs_bin = pad_bits(to_binary(abs(n)), 8)
    inverted = "".join("1" if b == "0" else "0" for b in abs_bin)
```

3. Sonuçları ekrana yazdır. (örnek çalışma):

*** Girdi = 25**

*** Binary: 11001**

*** Hex: 19**

*** Two's Complement (8bit): 00011001**

BİNARY

```
PS C:\Users\nailo\Desktop\python_temelleri> python -u "c:\Users\nailo\Desktop\
python_temelleri\proje.py"
Onluk tabanda bir sayi giriniz: 25
Binary için B, Hex için H yaziniz: B
Binary: 11001
Bellek (8-bit): 00011001
Kutucuklu gösterim: [0] [0] [0] [1] [1] [0] [0] [1]
PS C:\Users\nailo\Desktop\python_temelleri>
```

HEX

```
PS C:\Users\nailo\Desktop\python_temelleri> python -u "c:\Users\nailo\Desktop\
python_temelleri\proje.py"
Onluk tabanda bir sayi giriniz: 25
Binary için B, Hex için H yaziniz: H
Hex: 19
Bellek (8-bit): 00011001
Kutucuklu gösterim: [0] [0] [0] [1] [1] [0] [0] [1]
PS C:\Users\nailo\Desktop\python_temelleri>
```

TWO'S COMPLEMENT

```
PS C:\Users\nailo\Desktop\python_temelleri> python -u "c:\Users\nailo\Desktop\
python_temelleri\proje.py"
Onluk tabanda bir sayi giriniz: 25
Binary için B, Hex için H yaziniz: H
Hex: 19
Bellek (8-bit): 00011001
Kutucuklu gösterim: [0] [0] [0] [1] [1] [0] [0] [1]
PS C:\Users\nailo\Desktop\python_temelleri>
```

SONUÇ VE DEĞERLENDİRME

Öğrenilenler:

Projeden Öğrendiklerimiz

*** Bit/Byte kavramların**
*** Sayı sistemleri (Binary, Decimal, Hex)**
*** Two’s Complement yöntemi**
*** Python ile pratik uygulama**

```
┌──────────────────────────────────────────────────────┐
│   PROJEDEN ÖĞRENDİKLERİMİZ - ÖZET                    │
├──────────────────────────────────────────────────────┤
│                                                      │
│  🔵 BİT / BYTE KAVRAMLARI                            │
│     • Bit = En küçük veri birimi (0/1)               │
│     • Byte = 8 bit = 256 farklı değer                │
│     • 1 Byte = 1 ASCII karakter                      │
│     • Ağ hızı (Mbps) vs Dosya boyutu (MB) farkı      │
│                                                      │
│  🟢 SAYI SİSTEMLERİ                                  │
│     • Binary (2'lik): 1010₂ = 10₁₀                   │
│     • Decimal (10'luk): Günlük hayat                 │
│     • Hexadecimal (16'lık): 2F₁₆ = 47₁₀              │
│     • Dönüşüm algoritmaları                          │
│                                                      │
│  🔴 TWO'S COMPLEMENT                                 │
│     • Negatif sayı gösterimi                         │
│     • 3 adım: Ters çevir + 1 ekle                    │
│     • -128..+127 aralığı (8-bit)                     │
│     • Donanımda kolaylık (tek devre)                 │
│                                                      │
│  🟡 PYTHON İLE PRATİK                                │
│     • Hazır fonksiyon KULLANMADAN                    │
│     • Matematiksel döngüler (while, %, //)           │
│     • 8-bit bellek görselleştirme                    │
│     • Hata yönetimi ve test                          │
│                                                      │
│  📈 KAZANIMLAR:                                      │
│     ✓ Algoritmik düşünme                             │
│     ✓ Problem çözme becerisi                         │
│     ✓ Python programlama                             │
│     ✓ Donanım-yazılım ilişkisi                       │
│     ✓ Teorik bilgiyi pratiğe dökme                   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Projenin Katkıları:

*** Temel kavramların anlaşılması**
*** Pratik uygulama deneyimi**
*** Github ve video hazırlama becerileri**

```
┌──────────────────────────────────────────────────┐
│          PROJENİN KATKILARI - GELİŞİM GRAFİĞİ    │
├──────────────────────────────────────────────────┤
│                                                  │
│  📊 BİLGİ ve BECERİ GELİŞİMİ                     │
│                                                  │
│  Seviye: YÜKSEK ┤                                │
│          │    ▓▓▓▓▓▓▓▓▓▓▓▓▓▓ Python Uygulama     │
│          │    ▓▓▓▓▓▓▓▓▓▓▓▓▓  GitHub & Video      │
│          │    ▓▓▓▓▓▓▓▓▓▓▓    Sayı Sistemleri     │
│          │    ▓▓▓▓▓▓▓▓▓     Two's Complement    │
│  Seviye: ORTA  ├▓▓▓▓▓▓▓▓▓▓▓  Bit/Byte Temelleri  │
│          │                                       │
│  Seviye: DÜŞÜK ┤                                 │
│          └──────────────────────────────→        │
│            Başlangıç      Proje       Sonuç      │
│                                                  │
│  🎯 TEMEL KAVRAMLARIN ANLAŞILMASI:               │
│     • Bit/Byte fiziksel karşılığı                │
│     • Sayı sistemleri arası ilişki               │
│     • İşaretli/işaretsiz sayı farkı              │
│     • Bellek ve depolama mantığı                 │
│                                                  │
│  💻 PRATİK UYGULAMA DENEYİMİ:                    │
│     • Algoritma geliştirme                       │
│     • Python'da döngü mantığı                    │
│     • Hata yönetimi ve debug                     │
│     • Kullanıcı arayüzü tasarımı                 │
│                                                  │
│  🚀 GİTHUB ve VİDEO BECERİLERİ:                  │
│     • Version control (Git)                      │
│     • Repository yönetimi                        │
│     • README dokümantasyonu                      │
│     • Video sunumu hazırlama                     │
│     • Ekran kaydı ve edit                        │
│                                                  │
│  📈 GENEL KAZANIM:                               │
│     "Teorik bilgi → Pratik uygulama →            │
│      Sunum → Paylaşım" tam döngüsü               │
│                                                  │
└──────────────────────────────────────────────────┘
```

KAPANIŞ

DİNLEDİĞİNİZ İÇİN TEŞEKKÜRLER

NAİL BUĞRA ÖNÜR

24360859010

BİLGİSAYAR MÜHENDİSLİĞİ

BURSA TEKNİK ÜNİVERSİTESİ